

Contents

1	Databases Recap	4
1.1	The Relational Model	4
1.2	SQL	5
1.3	Relational Algebra	6
2	Conjunctive Queries	8
2.1	Conjunctive Queries	8
2.2	Computational Complexity	11
2.3	Acyclic Conjunctive Queries	19
3	Storage	23
3.1	Disk Space Management	23
3.1.1	Storage Hardware	23
3.1.2	Disk Space Manager	25
3.2	Buffer Management	26

3.3	File Management	29
3.3.1	Types of logical file	29
3.3.2	Record layout	33
3.3.3	Page layout	34
3.4	Storage Models	35
3.4.1	Query workloads	35
3.4.2	Storage models	36
3.4.3	Compression	37
4	Indexes	38
4.1	Types of Index	38
4.2	Tree Indexes	40
4.2.1	Tree Indexes	40
4.2.2	ISAM (Indexed Sequential Access Method)	41
4.2.3	B+ trees	42
4.3	Hash Indexes	45

5	Operators	47
5.1	Sorting	47
5.2	Aggregation	48
5.3	Joins	49
6	Query Planning	54
6.1	Plan Space	54
6.2	Scan Methods	55
6.3	Costing	57
6.4	Search	60
7	Transactions	62
7.1	Atomicity	62
7.2	Isolation	62
7.3	Isolation	65
7.4	Recovery	66
7.5	Distributed Transactions	67

1 Databases Recap

1.1 The Relational Model

- A database = a set of named relations
- A relation (table) consists of a schema (metadata) describing its structure and an instance (a collection of data that satisfies the schema)
- An instance is a multiset (bag) (or sometimes an ordinary set for convenience) of tuples (records/rows) made up of attributes (fields/columns)
- **Tuples are really maps (i.e. mathematical functions) from column names (or equivalently indices) to column values** rather than tuples in the mathematical sense, as the elements within a database tuple are not considered to be ordered

1.2 SQL

- DDL = data definition language
- DML = data manipulation language
- DCL = data control language (access control)
- SELECT keeps duplicates unless DISTINCT is added
- UNION/INTERSECT/EXCEPT remove duplicates unless ALL is added
- **A natural join is an INNER JOIN ON all columns that have the same name in both tables, and so is a [CROSS] JOIN if there are no shared columns** (as the universally quantified join condition has empty domain and so is vacuously true)

1.3 Relational Algebra

- SQL and relational calculus (SQL's formalisation) are declarative languages. Whereas relational algebra is an imperative language
- **Selection:** $\sigma = \text{WHERE}$ (careful, not SELECT)
- **Projection:** $\pi = \text{SELECT}$
- Renaming: ρ — only for the benefit of humans, computers can work quite happily referring to columns by indices
- Union: $\cup = \text{UNION}$
- Intersection: $\cap = \text{INTERSECT}$ — not needed for a universal set as $R \cap S = R - (R - S)$
- Set difference: $- = \text{EXCEPT}$
- Cross product: $\times = [\text{CROSS}] \text{ JOIN}$

- Join: $R \bowtie_{\theta} S = \sigma_{\theta}(R \times S) = R \text{ JOIN } S \text{ ON } \theta$ — *pedantically only a theta join if the condition is $a \circ b$ and $\circ \in \{\leq, \geq, <, >, =, \neq\}$ — if $\circ$ is $=$, it is an equijoin*
- $R \bowtie S = R \text{ NATURAL JOIN } S \approx$ an equijoin on all shared columns
- Group by: γ
- Duplicate elimination: δ
- Sorting: τ
- Division: $\div$

2 Conjunctive Queries

2.1 Conjunctive Queries

- Recall relational calculus:
SELECT airline FROM Flight AS F JOIN Airport AS Oa ON F.origin = Oa.code JOIN Airport AS Da ON F.destination = Da.code WHERE Oa.city = 'London' AND Da.city = 'Glasgow' $\Leftrightarrow$
 $\{z : \exists x, y : Airport(x, London) \wedge Airport(y, Glasgow) \wedge Flight(x, y, z)\}$
- Conjunctive queries = the $\{\exists, \wedge\}$ -fragment of relational calculus (no $\neg, \vee, \forall$)
= $\{\sigma, \pi, \bowtie\}$ -fragment of relational algebra/SQL with the σ condition restricted to $\{=, \wedge\}$
- A conjunctive query naturally lends itself to being written as a Prolog rule:
 $Q(x) \Leftarrow R_1(v_1), \dots, R_m(v_m)$ (note that x and v_i s are all tuples of variables rather than necessarily single variables)

- Evaluating conjunctive queries reduces to pattern matching the query body against the database in order to find all possible assignments of values to variables in the query that result in tuples in the database
- $\mathbf{terms}(A)$ = the set of terms used in an atom $A =$
 $\{t : t \text{ is a variable which occurs in } A\} \cup$
 $\{t : t \text{ is a constant which occurs in } A\}$
- A substitution from a set of terms S to a set of terms T is nothing more than a function $h : S \rightarrow T$
- A homomorphism from a set of atoms A to a set of atoms B is a substitution $h : \mathbf{terms}(A) \rightarrow \mathbf{terms}(B)$ which preserves structure ($h(A) \subseteq h(B)$ ($x \in A \rightarrow h(x) \in B$)) and does not affect (i.e. is the identity on) constants

- A match of a conjunctive query is a homomorphism from the body of the query to the database. The answer to a conjunctive query is the set of tuples which are the values of the distinguished (i.e. output) variables of the query under each match of the query

2.2 Computational Complexity

- By considering the decision problem corresponding to query evaluation rather than the search problem of query evaluation itself, we avoid having to compare the total complexity against the component attributable to simply outputting the result (which is naturally of varying size):
CQ-Evaluation = Given a database D , a conjunctive query Q , and a tuple $(a_1, \dots, a_k)$. Is $(a_1, \dots, a_k) \in Q(D)$?
- **Combined complexity of evaluation = complexity with query and data both part of the input**
- **Data complexity of evaluation = complexity when query is fixed in the problem itself** (rather than provided in the problem instance) **leaving only the database and the candidate tuple as inputs** — in real life, the database content evolves and the queries are constant, as described by data complexity

- **Proposition: CQ-Evaluation is NP-Complete in combined complexity but is PTIME in data complexity**

Proof:

$\in NP$:

Use a substitution $h : terms(body) \rightarrow terms(D)$ as a certificate. We need to check that h is a homomorphism and more specifically that h is a match of Q in D .

Verify that h is the identity on constants. Verify that $h(body) \subseteq D$.

Verify that $(h(x_1), \dots, h(x_k)) = (a_1, \dots, a_k)$ As D is an input, our certificate certainly is polynomial length and verifiable in polynomial time

$\in PTIME$ in data complexity:

Let n be the maximum cardinality of the domains of the columns of D . Let k be the number of variables in Q . If we were to enumerate hs as part of using the NP ness to build a decider, then we would have n^k to check and each one would take polynomial time to check. This is

polynomial time as Q and so k is fixed (and was exponential time when k was a characteristic of the input)

$\in NP\text{-Hard}$: $SAT \leq_P CQ\text{-Evaluation}$ (proof of this reduction is out of scope, but it is hopefully reasonably intuitively true)

- Problems that are given a query Q without a database are called static analysis problems. Because, as previously mentioned often queries are static and the database is evolving, being able to carry out static analysis would be very useful
- CQ-Satisfiability = Given a conjunctive query Q , does there exists a database D such that $Q(D)$ is non-empty? — if CQ-Satisfiability(Q) = reject, then there is no need for evaluation
- Proposition: CQ-Satisfiability $\in O(1)$ (and thus is pointless)
Proof: **We can construct a so called canonical database $D[Q]$ for Q . $D[Q]$ has a tuple for each clause of the body of Q**

wherein each distinct variable in x has been replaced by a special constant $\underline{x}$ associated to it (note the same value is reused when a variable is reused). By the construction of $D[Q]$ there is a match from Q to $D[Q]$ (the mapping used to create it is a bijection between variables in Q and constants in $D[Q]$, and so by extending it to accept constants and act as the identity on them it is a match).

We have shown that, for every Q , $Q(D[Q])$ is non-empty. Thus, for every Q , CQ-Satisfiability(Q)= Yes, and so accepting without reading the input would be a suitable decider!

- CQ-Equivalence = Given two conjunctive queries Q_1, Q_2 . Does $\forall D; Q_1(D) = Q_2(D)$ (abbreviate to $Q_1 \equiv Q_2$)?
- CQ-Containment = Given two conjunctive queries Q_1, Q_2 . Does $\forall D; Q_1(D) \subseteq Q_2(D)$ (abbreviate to $Q_1 \subseteq Q_2$)?
- *Containment is undecidable for the full relational algebra/SQL!*

- Proposition: CQ-Equivalence $\equiv_P$ CQ-Containment
 Proof sketch: $\wedge$ distributes over $\forall$. $Q_1 \equiv Q_2 \Leftrightarrow Q_1 \subseteq Q_2 \wedge Q_2 \subseteq Q_1$.
 $Q_1 \subseteq Q_2 \Leftrightarrow Q_1 \equiv Q_3$ where Q_3 is obtained by concatenating (i.e. conjuncting) the bodies of Q_1 and Q_2
- A query homomorphism from $Q_1(x_1, \dots, x_k) \Leftarrow body_1$ to $Q_2(y_1, \dots, y_k) \Leftarrow body_2$ is a homomorphism $h : body_1 \rightarrow body_2$ such that $(h(x_1), \dots, h(x_k)) = (y_1, \dots, y_k)$
- Lemma: Homomorphisms are closed under composition
 Proof: Trivial
- **Theorem (homomorphism theorem):** Let Q_1, Q_2 be conjunctive queries. $Q_1 \subseteq Q_2$ iff there exists a query homomorphism from Q_2 to Q_1 (be careful: direction of homomorphism is opposite to that of containment!)
 Proof: Omitted due to time

- Let QueryHomomorphism = Given conjunctive queries decide whether there exists a query homomorphism from Q_2 to Q_1 . Deduce that $(\text{CQ-Equivalence} \equiv_P) \text{CQ-Containment} \leq_P \text{QueryHomomorphism}$
- Proposition: QueryHomomorphism (and so CQ-Containment and so CQ-Equivalence) is NP-Complete

Proof sketch:

$\in NP$: Certificate = substitution. Verifier = verify is a query homomorphism

$\in NP\text{-Hard}$:

We have previously shown that $\text{CQ-Evaluation} \in NP\text{-Complete}$. It turns out that $\text{QueryHomomorphism} \leq_P \text{CQ-Evaluation}$ by constructing the canonical database for Q_1 . The inverse of the bijection that makes the canonical database can be composed on to the matching (if there is one) from the evaluation in order to obtain the needed query homomorphism; and if there is a query homomorphism then the bijection itself can be composed on to obtain a matching for the evaluation.

- **Query minimization by deletion:** If there exists an atom that can be removed from the query body without causing any output variable to no longer appear anywhere in the body and there is a query homomorphism to this new query, then update the query to this new query. Once there exists no such atom, we are done. variables to no longer appear anywhere in the body
- There is a loop invariant in query minimization by deletion of equivalence to previous query (and thus inductively to the original query) as we check for containment (due to the query homomorphism) of the new query in the old one and the other direction of containment is trivial (removing an atom weakens the body and so isn't going to have removed any tuples, we only needed to check it hadn't allowed new ones)
- It can be shown (proof out of scope) that any equivalent query has at least as large a body as that obtained by minimization by deletion, and thus

that **query minimization by deletion** (despite its greedy nature) is sufficient to obtain a query with the fewest possible atoms. Moreover, minimal conjunctive queries are actually isomorphic (the same up to variable renaming). **The result of query minimization is called the core of the query**

- *Although (most) static analysis and query evaluation are both NP-complete, static analysis has a smaller input. Thus, being able to do CQ-Equivalence (e.g. in order to optimize queries before CQ-Evaluation) in (at least not worse than thanks to the NPness) exponential time might actually be okay whereas in evaluation the database could well be large*

2.3 Acyclic Conjunctive Queries

- Some graph problems are NP-Complete for general graphs but actually are polynomial time for when the input is known to be drawn from a specific class of graphs (e.g. trees, planar graphs). The same applies to databases query evaluation, and we will actually represent queries as (hyper-)graphs
- The hyper-graph of a conjunctive query has vertex set of the terms of the body and has for each atom a set of of the vertices (a hyper-edge, which unlike a normal edge can have cardinality greater than 2) that are the terms used in that atom
- **A join tree of a hyper-graph of a conjunctive query is a tree with nodes labelled by hyper-edges such that: for every hyper-edge in the hyper-graph there is a node in the tree with that label and for every node in the hyper-graph the set of all hyper-edges that contain it is a connected subtree of nodes in the tree**

- A conjunctive query is called acyclic iff its hyper-graph has a join tree
- Proposition: It can be decided in polynomial time ($O((\text{number of nodes} + \text{number of hyper-edges})^2)$) whether a conjunctive query is acyclic

Proof (GYO-reduction algorithm):

- 1 Start with a join ‘‘tree’’ with a node for every hyper-edge and no edges in the tree
- 2 While there exists a node that occurs in at most one hyper-edge or there exists a hyper-edge that is a subset of another hyper-edge (note this may be because it is the empty set)
- 3 Remove that object from the hyper-graph
- 4 If removing a hyper-edge, add to the join ‘‘tree’’ an edge between it and its superset

5 If we have the empty hyper-graph
6 We will have a true join tree and so the
 hyper-graph will be acyclic
7 If we do not have the empty hyper-graph
8 There will exist no join tree (proof out
 of scope) and so the hyper-graph will be
 cyclic

- *There exists a linear time algorithm for acyclicity, but we only need to know the quadratic one above*

- $R \bowtie S = R \text{ SEMIJOIN } S = \pi_R(R \bowtie S) = \text{SELECT } * \text{ FROM } R \text{ WHERE NatJoinKey IN (SELECT NatJoinKey FROM } S)$
- Proposition: Let ACQ-Evaluation = Given a database D, an acyclic conjunctive query Q, and a tuple $(a_1, \dots, a_k)$. Is $(a_1, \dots, a_k) \in Q(D)$?
ACQ-Evaluation $\in PTIME$ (specifically $O(||D|| \cdot \log ||D|| \cdot ||Q||)$)
Proof (Yannakakis's Algorithm):
 - 1 Compute the join tree T of H(Q)
 - 2 Replace the node labels in T (hyperedges) with the set of tuples in the database for the relation that corresponds to that hyperedge
 - 3 In a bottom-up traversal of T carry out Parent SEMIJOIN Child
 - 4 Accept iff the result at the root is not the empty set (otherwise reject)
- $ACQ\text{-Evaluation} \in O(||D|| \cdot ||Q||)$ but that algorithm is out of scope

3 Storage

3.1 Disk Space Management

3.1.1 Storage Hardware

- A track = the path traced out by a disk head on a platter
- A cylinder = the paths traced out by all disk heads (when aligned) on their respective platters
- Seek time = the time taken for disk arm to find the correct track
- Rotational delay = the time taken for the target block of the track to be under the head

- Sequential IO = store contiguous data on the same track, then on other tracks in the same cylinder (switch to the next disk head), then start again in the adjacent cylinder (move each disk head one step)
- Sequential IO has no rotational delay within a track once it has found the start of its data and only switches tracks when necessary. Whereas, random IO incurs for every block seek time + rotational delay
- Solid state drives have fine-grained reads ($\approx 8\text{KB}$ pages) but coarse-grained writes ($\approx 1\text{MB}$ blocks)
- Write amplification of an SSD = writing small amounts of data (i.e. a few pages) requires erasing an entire block to rewrite — as each cell of an SSD can only be written a few thousands times, write amplification is a risk to longevity

- The difference between sequential reads and random reads on an SSD is negligible. Whereas, **even on an SSD sequential writes are noticeably faster than random writes** (as random writes lead to more write bandwidth being wasted on write amplification)

3.1.2 Disk Space Manager

- A DBMS' disk space manager obtains one or more large files **on disk** (relying on the filesystem to store it as contiguously as possible) **then manages the location of database pages within this file itself** (aiming to store “nearby” pages “near” to each other in the file (and thus on disk))
- Multiple database files are required if the database is to be spread over multiple disks: the disk space manager maintains a bijection between page IDs and tuples of file name and offset within the file

3.2 Buffer Management

- A buffer pool = an in-memory cache (of disk pages). The buffer pool is partitioned into frames each of which is either empty or holding a disk page
- The buffer pool provides an abstraction to the higher levels of the DBMS that the database is stored in RAM (albeit this leaks as we allow the planner to tell the buffer pool what to evict and what not to evict)
- **Pin(pageID):** If it is in the buffer pool, return a pointer to its frame. If it is not in the buffer pool, load it into the buffer pool and return a pointer to its frame
- **Unpin(pageID, dirtied):** Once every pin of this page has unpinned, it is eligible for eviction. If a page is dirtied (on any unpin) it has been modified and so should be flushed to disk upon eviction

- **Fix(pageID):** Evict only after all non-fixed pages
- **Hate(pageID):** Evict before all non-hated pages
- The buffer manager maintains a $\text{HashMap}\langle \text{PageID}, \text{FrameID} \rangle$, and an $\text{Array}\langle \text{Tuple}\langle \text{IsDirty}, \text{PinCount} \rangle \rangle$ indexed by frameID
- LRU (least recently used) eviction policy = evict the entry that was touched the longest ago (e.g. maintain a priority queue based on most recent access time) — careful: not least frequently used!
- **The CLOCK eviction policy** approximates LRU without the overhead of having to find the minimum: **Consider the frames to be arranged in a circular buffer (“clock”).** Maintain a pointer (“clock hand”) to the next frame to consider evicting the page of. Increment pointer (rotate clock hand) until (i.e. always at least one step) a frame that has pin count 0 is found, mark this “warned”. Have the pin function unset the

“warned”, and have empty frames initialised as “warned”. Continue in this fashion (potentially “warn”ing more frames) until a frame with pin count 0 and “warned” already true is found, evict

- LRU/clock is good for random access, but counter-intuitively **most recently used (MRU) is optimal for repeated sequential scans (e.g. a join)**. This is because MRU has a **stable prefix of the table in the cache** whereas LRU is cyclical with the scan (and so evicts what is old but will be read again soon in order to keep what is new but won't be re-read until it has aged to have also been evicted)

3.3 File Management

3.3.1 Types of logical file

- Records are packed into pages which are cached in memory by the buffer manager. Pages are packed into logical files in order to assist the storage manager in storing them in physical files sensibly
- I/O-only cost model: P = number of data pages in file, R = number of records per page, D = average time to read/write a disk page
- **A heap file stores records in no particular order (unlike a heap data structure!) — begins with a page directory (header pages storing metadata (e.g. pointer to and amount of free space in) the data pages)**
- **Scan all records in any file = $P \times D$** (we have to read all the pages to produce the output of this query)

- **Search on key in a heap file** = $\frac{P}{2} \times D$ **on average** (we know there will be at most one hit)
- **Search on non-key in a heap file** = $P \times D$ (we have to check all records to be sure to get all hits)
- **Range search in a heap file** = $P \times D$ (we have to check all records to be sure to get all hits)
- **Insert record into a heap file** = $2D$ (read last page (1D) + append new record to page (0) + write updated page back to disk (1D))
- **Delete record from a heap file** = $(\frac{P}{2} + 1) D$ **on average** (search on key to find page containing record ($\frac{P}{2} \times D$ on average) + delete record from page (0) + write updated page back to disk (1D))

- A sorted file stores records in a particular order
- Search on key in a sorted file if sorted by search attribute = $\log_2(P) \times D$ in worst case (we can now use binary search instead of linear search)
- Search on non-key in a sorted file if sorted by search attribute = $(\log_2(P) + \text{number of hits})D$ (all hits will be contiguous)
- Range search in a sorted file if sorted by search attribute = $(\log_2(P) + \text{number of hits})D$ (all hits will be contiguous)
- Any search in a sorted file if sort attribute is not search attribute = cost of that search in a page file (we have to go back to linear search)
- Insert record into a sorted file = $(\log_2 P + P)D$ on average
search for insert point ($\log_2 P \times D$ on average) + shift subsequent pages to

make gap at insert point (read half the pages on average + write half the pages on average = $P \times D$ cost on average)

- **Delete record from a sorted file = $(\log_2 P + P)D$ on average**
search for record ($\log_2 P \times D$ on average) + shift subsequent pages to fill gap
- **An index file stores records (or pointers to records) so that they can be efficiently searched**

3.3.2 Record layout

- **Fixed-length record layout** = Let every field have a fixed length (e.g. place upper bounds, which we can afford to always hit, on the size of each field). Fields can be accessed by pointer arithmetic — a metadata bitmap is needed to track whether nullable fields are null (specifically, field storing empty string and empty field due to null are indistinguishable otherwise)
- **Variable-length record layout** = Store an array of field offsets in a header alongside the isNull bitmap
- It is common to use fixed-length record layout for everything. This is achieved by replacing the true data of variable-length fields with a tuple (offset, length) and storing the true data at that offset at the end of the record

3.3.3 Page layout

- **Slotted-pages page layout** = Maintain a slot directory: a mapping from slotID (recordID = (pageID, slotID)) to offset of start of slot (i.e. start of record) — the slot directory provides indirection which allows pages to be de-fragmented (VACUUM in PostgreSQL) within a page without having to change record IDs

3.4 Storage Models

3.4.1 Query workloads

- On-Line Transactional Processing (OLTP) = Simple queries that read and write small amounts of data per transaction — needs to be fast
- On-Line Analytical Processing (OLAP) = Complex read-heavy queries that read large amounts of data to compute aggregates — doesn't need to be fast but we should still optimize for it if we know it is common
- Hybrid Transactional and Analytical Processing (HTAP) = the same database instance needs to support both OLTP and OLAP without them getting in the way of each other i.e. we want to run data analytics on the primary without disrupting user-facing services

3.4.2 Storage models

- **Row storage** = what we would normally think of = store all attributes of a tuple contiguously (and thus in the same page) — good for OLTP (as we can use the index to jump to the row we are inserting/updating/deleting then access any and all attributes we need)
- **Columnar storage** = store each attribute for all tuples contiguously (and thus in the same page). This can be achieved by storing each attribute as an array of fixed-length values indexed into by the tuples and using dictionary compression to convert any variable-length data into fixed-length (assign an identifier to each unique value then store a mapping from identifiers to values elsewhere) — good for OLAP (as OLAP scans down particular attributes for lots of rows to aggregate them)

- Partition Attributes Across (PAX) = In order to combine the benefits of columnar and row, horizontally partition tuples into row groups and vertically partition attributes of row groups into column chunks

3.4.3 Compression

- Compression not only decreases disk/RAM capacity requirements but also allows more data to be transferred for the same IO cost. It does though increase CPU cost
- **Categorical data can be compressed by run-length encoding, sorted numerical data can be compressed by delta encoding, frequent strings can be compressed by dictionary encoding**

4 Indexes

4.1 Types of Index

- The (search) key of an index is not necessarily the same as the (primary) key of the relation (although the primary key definitely should be one set of attributes for which an index is created). In particular, a search key does not have to uniquely identify tuples
- **Primary/unique index = search key is a superset of the primary/a candidate key** — searches will match at most one tuple
- **Secondary index = searches may match more than one tuple**
- An index can store the records themselves (acts as a sorted file) (only used for primary key to avoid wasting storage), or store a reference to the location of the records in a heap file (used for additional indexes) —

typically store records twice: once in a sorted file (primary index) and once in a heap file (but only one heap file for all additional indexes)

- **Clustered index = “near” search keys have “near” references**
— range search has to read fewer pages to get the records it needs, and the pages it needs have spatial locality
- Unclustered index = is not a clustered index (e.g. keys are approximately uniformly distributed across pages)
- **Index of primary key is vacuously clustered as this is a sorted file rather than references to the heap**
- We can only guarantee clustering of one additional index as we only have one heap file to (approximately) sort one way

4.2 Tree Indexes

4.2.1 Tree Indexes

- A tree-structured index can be viewed as an index for an index ... for an index
- By using wide trees, we can have a much larger base on our logarithm than binary search in a sorted file (or equivalently a binary search tree).
Moreover, a greater amount of similar data is in the same disk block
- When working with string keys, we can optimise search trees by making them trie like (only storing the prefix that characterises that level)

4.2.2 ISAM (Indexed Sequential Access Method)

- In ISAM, “non-leaf” pages (used to direct the search) are immutable, only “leaf” pages (data pages) are manipulated
- ISAM “non-leaf” node structure: $\langle P_0, K_1, P_1, \dots, K_m, P_m \rangle$ where P_0 is a pointer to the far-left subtree and that subtree contains keys up to K_1 , P_1 is a pointer to the subtree of keys k that obey $K_1 \leq k < K_2$, ..., P_m is a pointer to the keys that are at least K_m
- ISAM “leaf” node structure: $\langle O, K_1, P_1, \dots, K_m, P_m \rangle$ where each P_i is a pointer to the location in the data file that contains key K_i and O is a pointer to an overflow leaf page (which itself may point to another overflow leaf page ...) for further equalities that lie inside the range of the parent

- ISAM has the same issue as a binary search tree without a balancing algorithm that the ordering of insert operations can cause the height of the tree to grow unnecessarily

4.2.3 B+ trees

- **B+ trees are self-balancing: All leaves are at the same depth**
- *b-order of a B+-tree (branching factor) = number of direct children permitted to each node (the same for all nodes)*
- **The node structure of B+ trees is essentially the same as ISAM** (but don't need the overflow pointer in leaves any more), but the insert algorithm is no longer straight forward due to the need to rebalance
- Order (d) = the value such that there are $2d$ K s (and thus $2d + 1$ (max fan-out) P s) available to each node

- **Search:** Start at root. At each level, use binary search to find the child whose interval contains the value we are looking for, move to this node and repeat. Upon reaching a leaf, do a binary search of its elements, if we find our key, we can follow the pointer to the record; if we don't, there is no such record
- **Insert:** Use the search algorithm to find the leaf and the insert point within the leaf's elements which will maintain the ordering of keys within the leaf. If leaf has no remaining capacity, split current node before the insert then reattempt the insert (it will now succeed)
- **Node split algorithm:** Find the median (fast as already ordered). Place the elements less than the median in one new node and the elements at least than the median in the other new node (which side the median goes to is arbitrary). We no longer need the node we split. Incorporate the median

into the range of the parent of the node we split (“copy up” at leaf, “push up” at non-leaf). If this overflows, repeat this procedure. If we overfill the root, imagine it has an empty parent (i.e. make a new root that splits the data into 2 ranges based on the median of the present root) — by pushing up towards the root the growth needed to maintain the branching limit, balance is implicitly maintained

- Delete: Search for the node. Remove the element. We are done — *in general, $B(+)$ -trees are meant to have every node at least half-full so that they do not have unnecessary depth, and there is a "redistribution" algorithm for after a delete to ensure this. However, in databases we often don't bother with this as we are typically happy to burn some disk space in order to save IO time by having free space for future inserts*

4.3 Hash Indexes

- **Static chained hashing:** Allocate a fixed number N of successive pages to form primary buckets. Each bucket has capacity for a certain number of index entries (as before, either records themselves or pointers to records) **plus a pointer to an overflow page** (which may store a pointer to another overflow page...). Each bucket consists of the primary bucket page and all the overflow pages connected to it. A hash function maps a key to a primary bucket page and the whole bucket can then be traversed to find the desired page
- **Extensible hashing** allows the number of buckets to be expanded (and thus overflow chains to be eliminated) without the need to rehash all the pages: Use the n (global depth) least significant bits of the hash value to index into a directory to find a pointer to a bucket. For search/delete,

scan the bucket until a matching key is found (for delete may be able to decrease global depth)

- Local depth (d) of a bucket in extensible hashing = the number of least significant bits which all hash values in a given bucket agree on — for correctness n must be at least (for efficiency will be exactly) the greatest d
- Insert into a bucket with capacity in extensible hashing: Straightforward
- Insert into a full bucket in extensible hashing: Split the bucket potentially causing the local depth of this bucket (and the new one) to increase. Update the directory (increase global depth if necessary)

5 Operators

5.1 Sorting

- Sorting is not only for ORDER BY. It can also be a way of implementing DISTINCT, GROUP BY, (equi-)JOIN
- External sorting refers to the use of sorting algorithms that are designed around the cost of disk IO when sorting data which exceeds the available main memory space
- **$(B - 1)$ -way external merge sort ($B \geq 2$):** Firstly read B pages at a time into memory, sort data within this group, then write this run of sorted data back to disk. We end up with $\lceil \frac{N}{B} \rceil$ runs each of size B . Then stream groups of $B - 1$ runs using $B - 1$ buffer pages in order to merge them into a stream of output using one new buffer page. Recursively merge until there is only one run

- External merge sort requires $1 + \lceil \log_{B-1} N \rceil$ passes and $2N(1 + \lceil \log_{B-1} N \rceil)$ disk IOs

5.2 Aggregation

- Unless there is a subsequent operator that will also benefit from the data being sorted, GROUP BY and DISTINCT are typically more efficient by hashing than by sorting
- **External hashing:** We are given a budget of B buffer pages (much like external sorting). Use a hash function h_p to partition the data into $B - 1$ partitions. Use the remaining page to stream data in from disk. In each partition use a different hash function h_r to create a hash table for the partition
- Data with the same key will always be mapped to the same slot of the same partition, this is helpful. Data with different keys will sometimes be

mapped to the same slot of the same partition (hash collision), which is not desirable: we can use open chaining (store a pointer to the head of a linked list at the slot and scan the keys of the list) for collision resolution

- DISTINCT: Hash keyed on the SELECT attributes. If data arrives with a duplicate hash key, discard the duplicate
- Aggregation functions: Hash keyed on the GROUP BY. If data arrives with a duplicate hash key, reduce the current value and the new value into the next current value using the aggregation function (for AVG track COUNT and SUM as we go along, then use these to compute the mean at the end)

5.3 Joins

- Let table R have M pages and m tuples (deduce $M < m$), and table S have N pages and n tuples (deduce $N < n$)
- Naive join: $R \bowtie_C S \equiv \sigma_C(R \times S)$ — never use this one!

- **Simple nested loops join:** for page $P_r \in R$ for tuple $r \in P_r$ for page $P_s \in S$ for tuple $s \in P_s$ emit if (r, s) satisfies **join condition** — also never use this one! — repeats a full scan of S for every tuple in R (we are likely to lose P_r from the buffer while scanning S , and so have to reload it each time we need to get the next tuple $r \in P_r$)
- **Cost of simple nested loops join** = $M + mN$
- **Block nested loops join:** for block of $B - 2$ pages $B_R \in R$ for page $P_s \in S$ for tuple $r \in B_R$ for tuple $s \in P_s$ emit if (r, s) satisfies **join condition** — repeats a full scan of S for every block of $B - 2$ pages from R
- **Cost of block nested loops join** = $M + \lceil \frac{M}{B-2} \rceil N$ — if $M \leq B - 2$, this collapses to $M + N$ which is theoretically optimal as it is the amount of output that would have to be produced if the join condition accepted every tuple (but that is not very realistic)

- Page nested loops join = block nested loops join with $B=3$
- If we have a suitable index for the inner table, we don't need to scan it at all to find tuples that match the outer table
- If we are doing an equijoin we can use a sort-merge join or a hash join instead of a nested loops join
- Sort-merge join:
 - 1 sort R on join key A (e.g. external merge sort) to obtain R_sorted
 - 2 sort S on join key A (e.g. external merge sort) to obtain S_sorted
 - 3 r =: head(R_sorted)
 - 4 s =: head(S_sorted)
 - 5 while r $\neq$ EOF and s $\neq$ EOF:
 - 6 if r.A > s.A then s = s.next
 - 7 else if r.A < s.A then r = r.next

```
8      // r.A = S.A
9      else then { emit (r, s); s = s.next } //
      this does not handle the edge case of
      duplicates in R.A
```

- If all tuples match each other, sort-merge join degenerates to simple nested loop! But if every tuple has at most one match, then the merge cost is the optimal $M + N$ (and we won't need to do the sorting if we get sorted data from a previous operator)
- **Hash join:** Build a hash table for the outer relation R keyed on the join key. Scan the inner relation S and for each tuple $s \in S$ use the hash table for R to jump to matching tuples
- Hash join has the optimal IO cost $M + N$ (if the hash table fits into RAM) as each tuple is scanned exactly once

- **GRACE hash join:** Use a hash function to create partitions in which all relations with the same value of the join key are in the same partition. Write partitions to disk temporarily. One partition at a time (i.e. a pair of R and S for the same partition hash value) create a hash table (note can discard the current hash table once we move onto the next partition) to carry out the join for that partition
- **IO cost of GRACE hash join is $3(M + N)$** (scan data to create portions + write partitions to disk + read portions for hash join)

6 Query Planning

6.1 Plan Space

- The query rewriter uses rules that do not rely on the relation state (e.g. remove predicates that are always true/false, remove tables from which no attributes are used)
- The query optimizer works with the output of the query rewriter to enumerate plans and uses a cost estimation system to decide which plan to follow based on statistics about the relation state
- Plan space = which plans to consider — trade-off of time spent on search of optimization and time gained on execution from optimization
- The order in which tables are joined (as $\bowtie$ is commutative and transitive the order does not affect the output) can have a big impact on performance

- **Projection push-down:** Project away as soon as possible columns that are not needed by subsequent operators — only makes sense for row stores, not column stores
- **Selection push-down:** Apply σ as soon as possible e.g. apply part of a predicate before a join
- **Predicate pushdown:** Apply the most selective predicate in a conjunction first

6.2 Scan Methods

- Logical equivalences create equivalence classes of query trees by reordering (or rewriting) the relation algebra operators themselves, but there are also physical equivalences for how to label the tree nodes with how to execute them (e.g. which join algorithm to use)

- Sequential scan: Iterate through pages and iterate through tuples in each page — we can always do this
- **Hash indexes only support index scans for equality predicates (on the entire key), whereas a tree index supports index scans for so called lexicographic predicates** (for some prefix of the key (thus the ordering of columns within the key matters) apply equality predicates to all columns except the last in the prefix which can possibly be inequality/between)
- Index-only scan: Query only uses search key attributes, so it isn't even necessary to retrieve the data page itself from the pointer keyed to by the index (the key itself contains the information we need) e.g. taking an average of a range of a column for which there is a tree index — more precisely, for such scans unclustered indexes behave like clustered indexes
- **Bitmap scan: If a predicate can be partitioned into a conjunction/disjunction of predicates for which there are**

different indexes, retrieve the sets of matches from each index then take intersection/union

- If there are additional clauses conjuncted on that we don't have indexes for, we can still use a bitmap scan and select by the additional clauses as we scan
- If there are additional clauses disjuncted on that we don't have indexes for, we cannot use an index scan (have to use a sequential scan and select by the entire predicate)

6.3 Costing

- **NKeys = number of distinct values in a column**
- **Selectivity (aka reduction factor) of a predicate = $|\text{output}| / |\text{input}|$** — careful: higher selectivity means the filter drops fewer tuples!

- Assuming that data is uniformly distributed, $sel(A = i) = \frac{1}{NKeys(A)}$
- **Selectivity follows laws of probability:** $sel(\neg P) = 1 - sel(P)$; if $P1, P2$ are independent then $sel(P1 \wedge P2) = sel(P1) \cdot sel(P2)$; $sel(P1 \vee P2) = sel(P1) + sel(P2) - sel(P1 \wedge P2)$
- $sel(A = B)$ (e.g. an equijoin) $= \sum_i sel(A = i \wedge B = i) = \frac{1}{\max(NKeys(A), NKeys(B))}$ (under the assumption that columns A and B are independent)
- If R and S are (equi-)joined on a column A which is a foreign key in S referencing R, then $\pi_A(S) \subseteq \pi_A(R)$ and so $|R \bowtie S| \leq |S|$ (exact equality if A is not nullable in S)
- If R and S are (equi-)joined on a column A that is not a key for either table, $|R \bowtie S| = sel(R.A = S.A) \cdot |R| \cdot |S| = \frac{|R||S|}{\max(NKeys(R.A), NKeys(S.A))}$

- To make accurate estimates for non-uniformly distributed attributes (i.e. most data), we need the database to track more statistics than just NKeys
- **A histogram is a partition of the domain into intervals (buckets) with counts in each bucket of the number of tuples and the number of distinct values — we still approximate uniformly within each bucket but at least we now do not effectively have a single bucket for the entire distribution**
- Equi-width histogram: Every interval has the same width (i.e. number of distinct values)
- Equi-depth histogram: Every bucket contains approximately the same number of tuples — motivation: Have the most accuracy on the most frequently occurring data (as this is likely to be queried most often) — corollary: equi-width is better for the low frequency data (but this is likely queried less often)

6.4 Search

- Cost of using a B-tree index I to lookup a primary key is $\text{Height}(I) + 2$ (+2 is from + 1 to reach the leaf, + 1 to follow the reference in the leaf)
- Cost of using a clustered index I for a selection in a relation R is asymptotically $[NPages(I) + NPages(R)] \cdot selectivity$
- Cost of using an unclustered index I for a selection in a relation R is asymptotically $[NPages(I) + NTuples(R)] \cdot selectivity$
- Cost of using a sequential scan for a selection in a relation R is $NPages(R)$
- There are $(2N-2)!/(N-1)!$ equivalent join orderings, it is completely infeasible to enumerate all

- Multi-table plans have optimal substructure, so **we can use dynamic programming to create single-table plans** (as described above) then **2-table plans** from these then **3-table plans** from these and so on until we have a plan for our full query:

$$OPT(R \bowtie S \bowtie T) = \min(OPT(R, S) \bowtie T, OPT(R, T) \bowtie S, OPT(S, T) \bowtie R)$$

7 Transactions

7.1 Atomicity

- Shadow paging (copy-on-write): Make copies of pages for transaction to modify then replace pages with the copy upon commit
- Logging: Log actions, use log to know how to reverse actions upon abort — popular in industry as will want an audit log anyway

7.2 Isolation

- Pessimistic concurrency control = Don't allow operations that have the potential to conflict to be interleaved (i.e. use locking)

- Optimistic concurrency control = Deal with conflicts as and when they happen (i.e. wait until commit then rollback if a conflict actually occurred)
- Schedule for a set of transactions = a sequence that contains the steps (read/write table named ...) of all of the transactions and in which the ordering of the steps in each transaction relative to the others in that transaction has been preserved
- We can define an equivalence relation over transactions based on bringing the database into the same state as each other for every initial state and for every possible combination of record(s) touched by the steps of the transaction
- Serial schedule = a schedule in which transactions have not been interleaved

- Serializable schedule = a schedule which is equivalent to a serial schedule
- Schedules $s1$, $s2$ are conflict equivalent iff for every pair of conflicting actions $p1$, $p2$; $p1s1p2 \Rightarrow p1s2p2$ and $p2s1p1 \Rightarrow p2s2p1$
- A schedule is conflict serializable iff it is conflict equivalent to a serial schedule iff it can be transformed into a serial schedule by swapping consecutive non-conflicting operations
- A dependency graph can be created to represent the conflicts in a schedule. A schedule is conflict-serializable iff its dependency graph is acyclic (a conflict-equivalent serial schedule can be obtained by a topological sort of the graph)
- A schedule is view-serializable iff it is conflict-serializable or is only not conflict-serializable due to “blind writes”

7.3 Isolation

- S-lock = shared lock (reader)
- X-lock = exclusive lock (writer)
- **2-phase locking:** A transaction can request locks iff it has not yet released any locks. Phase 1 (growing): Transaction requests locks. Transaction cannot release locks. Phase 2 (shrinking): Transaction releases locks. Transaction cannot request locks
- **Strict 2-phase locking:** X-locks are not released until commit/abort — prevents cascading aborts as other transactions are not able to read uncommitted writes
- **Deadlock detection:** Periodically create “waits-for” graph and check for cycles, if a cycle is found chose a transaction to abort

- **Deadlock prevention:** If a transaction tries to acquire a lock held by another transaction, consider whether we need to abort a transaction to avoid a cyclic wait in the future. Can either use wait-die policy or wound-wait policy
- **Wait-die:** If requester is older it waits for holder, otherwise the requester “kills itself”
- **Wound-wait:** If the requester is older than the holder then the requester “murders” the holder, otherwise the requester waits

7.4 Recovery

- The buffer pool manager is responsible for managing the trade-off between performance and durability

- **Write-ahead logging:** Information needed to describe a change (and thus to “redo” or “undo” it) is recorded in an append-only log before the change is made
- Associated WAL records must be flushed to disk before a “steal” (eviction to disk of pages dirtied by an uncommitted transaction) or a “no force” (marking of transaction as committed but buffering flush of dirtied data pages to disk) but these operations are made possible by the existence of the WAL

7.5 Distributed Transactions

- **2-phase commit:** Phase 1 prepare (try to get unanimous decision to commit), phase 2 accept (disseminate result of vote). Each node tells coordinator whether it wants to commit or abort based on its local state; coordinator tells each node to commit iff no node

wanted to abort (ie tells every node to abort if even one node asked to abort); nodes acknowledge receipt of this consensus decision

- **To facilitate crash recovery, nodes should log their vote to their local disk before sending their vote and coordinator should log consensus decision to its local disk before multicasting it. Then each node should log its receipt of the consensus decision to their local disk between actioning it and acking the message, and coordinator should log receipt of ack from each node**
- **Ordering of messages is necessary for correctness (e.g. use a logical clock)**
- If a node involved in the transaction is unresponsive in submitting a vote, the transaction will timeout and be aborted by the coordinator (*to be able to make progress we would need full Paxos/Raft*)

- If a node voted but is unresponsive to the result, the coordinator waits for it and when it recovers it will ask the coordinator what the consensus was (which the coordinator will be able to find from its logs)
- If a node voted but is unresponsive to the result, the coordinator waits for it and when it recovers it will ask the coordinator what the consensus was (which the coordinator will be able to find from its logs)
- If a node cannot reach the coordinator to ack the consensus or does not receive a consensus from the coordinator, the node waits for it and the coordinator when it recovers will resend the (same, from its logs) consensus